अर्किओन
INTERNATIONAL INSTITUTE OF MEDICAL SCIENCE & TECHNOLOGY COUNCIL

A Mini Project Report on

SMART CALCULATOR

Submitted by : **COHORT – 02**

Awad Zuhair (1HK22EC019)
Anush Franklyn F (1RG22EC003)
Keerthana G (4VM22EC024)
Manoj T (1AM22EC060)
Mohammed Maaz (1HK22EC094)
Murugesh M R (1AM22EC067)
Neha Metri (1BI22ET032)
Raphael J S (1RG22EC028)
Sanjana Balaramsingh Hajeri (1BI22ET043)
Shrinath D Pol (4AD23EC409)
Sushankitha S (1BI22ET053)
Tejashwini N M (4VM22EC053)
Vidya Sagar T J (1KI22EC118)

ABSTRACT

The Smart Calculator is an embedded systems mini project designed using Arduino UNO, a 4x4 matrix keypad, and a 16x2 LCD display. The system performs fundamental arithmetic operations including addition, subtraction, multiplication, and division, along with advanced functions such as percentage calculation, factorial computation, and exponentiation (power function).

The project supports **Sustainable Development Goal 4 (Quality Education)** by providing an affordable, hands-on learning tool that integrates mathematics, programming, and electronics. The calculator enhances logical reasoning, embedded programming skills, and practical STEM education exposure.

The system is implemented using C++ in the Arduino IDE and demonstrates real-time input processing and output display through an LCD interface. Error handling such as division by zero and overflow protection in factorial computation ensures system reliability.

.

INTRODUCTION

Project Overview

In the modern educational ecosystem, practical learning tools are essential for improving conceptual clarity in mathematics and engineering. Traditional calculators perform arithmetic tasks but do not provide insight into internal computational logic.

The Smart Calculator project bridges this gap by building a hardware-based calculator system using Arduino. It allows students to understand how mathematical operations are implemented programmatically and executed via microcontrollers.

The system consists of:

- Arduino UNO (processing unit)
- 4x4 Matrix Keypad (input interface)
- 16x2 LCD with I2C module (output display)

The calculator supports:

- Addition
- Subtraction
- Multiplication
- Division
- Percentage
- Factorial
- Power (Exponentiation)

Project Objectives

1. To design a functional calculator using embedded hardware.
2. To implement mathematical algorithms in Arduino C++.
3. To demonstrate keypad scanning and LCD interfacing.
4. To integrate advanced functions such as factorial and power.
5. To support SDG 4 by promoting interactive STEM learning.
6. To develop error-handling mechanisms in embedded systems.

BLOCK DIAGRAM

System Components

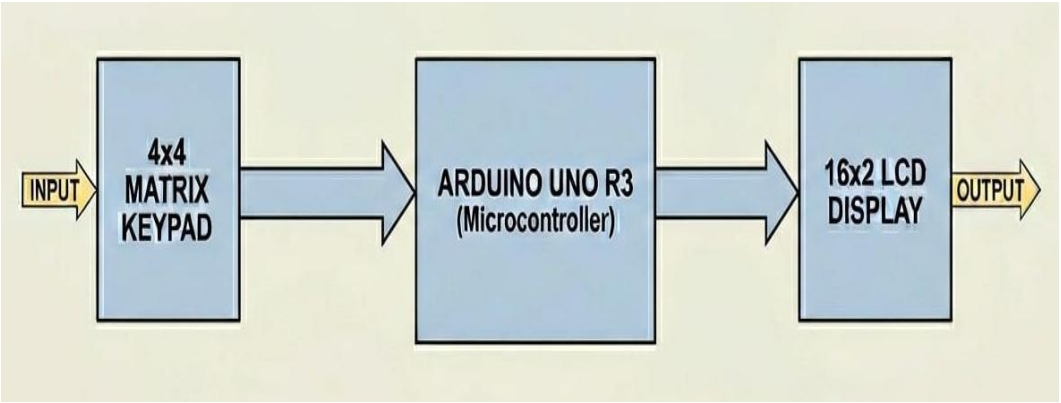


Fig: Block Diagram

Block Diagram Explanation

The keypad acts as the input module where users enter numerical values and select operations. The Arduino UNO processes the input, executes the corresponding mathematical function, and sends the result to the 16x2 LCD display.

The LCD displays real-time input and final output results. The entire system is powered through USB or an external 5V supply.

ALGORITHM & CODE LOGIC

Algorithm

Step 1: Initialization

- Initialize LCD.
- Configure keypad matrix.
- Display welcome message.

Step 2: Input Capture

- Continuously scan keypad.
- Store digits in `num1` or `num2`.

Step 3: Operator Detection

- A → Addition
- B → Subtraction
- C → Multiplication
- D → Division / Power toggle
  → Percentage / Factorial
  → Equal

Step 4: Processing

- Convert string inputs to float.
- Perform selected operation.
- Handle special cases:
  - Division by zero.
  - Factorial overflow (>12).
  - Power mode activation.

Step 5: Output Display

- Display result on LCD.
- Reset variables for next calculation.

System Overview

The calculator runs as an infinite polling loop on an Arduino microcontroller. It reads keypad input, maintains a finite-state machine, and renders results to a 16x2 I2C LCD display. All logic is driven by keypress events — there is no background processing between keypresses.

Data Structures & State Variables

Variable	Purpose
num1[16]	Character buffer storing the first operand (up to 14 digits + null)
num2[16]	Character buffer storing the second operand
idx1, idx2	Cursor indexes tracking how many digits have been entered into each buffer
op	Current operator: 0=none, A=+, B=-, C=x, D=/, P=power, S=sqrt, R=percent
Current Mode	Enum: NORMAL, POWER, PERCENT, SQRT — controls which keys are active
Factorial Mode	Boolean flag set when * is pressed with no operator selected
Result Displayed	Boolean flag — locks all input after a result is shown
First A Pressed	Boolean flag for detecting double-A reset sequence

Key Routing Logic

The following decision tree describes how each keypress is handled:

Digit Keys (0-9)

- If op == 0: append digit to num1 buffer, print to LCD
- If op != 0: append digit to num2 buffer, print to LCD
- Maximum 14 digits enforced per buffer (idx < 14 check)

Key A - Addition / Reset Detection

- If part of double-A sequence: triggers showReset() and returns
- If currentMode == NORMAL and not a reset: sets op='A', prints '+' to LCD

Key B - Subtraction

- If currentMode == NORMAL: sets op='B', prints '-' to LCD

Key C - Multiply / Power Mode

- If C is pressed and op is already 'C' (second press): activates POWER mode, calls showMode("Power Mode"), resets buffers
- If currentMode == POWER: sets op='P', prints '^' to LCD (the exponent separator)
- Otherwise (first C press, normal mode): sets op='C', prints 'x' to LCD

Key D - Divide / SQRT Mode / Percent Input

- If D is pressed and op is already 'D' (second press): activates SQRT mode, calls showMode("SQRT Mode"), resets buffers
- If currentMode == SQRT: sets op='S', prints 'sqr' to LCD
- If currentMode == PERCENT: sets op='R', prints '%' to LCD
- Otherwise (first D press, normal mode): sets op='D', prints '/' to LCD

Key * - Factorial / Percent Mode Trigger

- If op == 'C' and currentMode == NORMAL: switches to PERCENT mode, shows "Percent Mode", restores num1 to display
- If op == 0 and currentMode == NORMAL: sets factorialMode=true, prints '!' to LCD

Key # - Calculate

This is the execution trigger. The following sub-algorithm runs when # is pressed:

```
a = atof(num1) // convert string buffer to double
b = atof(num2)
IF factorialMode:
   IF a > 12 → showError("Overflow")
   ELSE → result = factorial ((unsigned long) a) → printResult()
ELSE IF op == 'S':  // SQRT
   result = sqrt(a) → printResult()
ELSE IF op == 'P':  // Power
   result = round (pow (a, b)) → printResult()
ELSE IF op == 'R':  // Percent
   result = (a * b) / 100.0 → printResult()
ELSE:  // Normal ops
   '+' → result = a + b
   '-' → result = a - b
   'x' → result = a * b
   '/ ' → if b==0: showError("Div by 0") else result = a / b
       → printResult()
```

Mode Transition Diagram

Transition	Result
Startup	currentMode = NORMAL
C pressed twice (no prior input)	currentMode = POWER
D pressed twice (no prior input)	currentMode = SQRT
num1 entered, then C then *	currentMode = PERCENT
# pressed (any mode)	Result displayed, input locked
A pressed twice (any state)	Full reset to NORMAL

Error Conditions

Error	Cause	Recovery
Div by 0	Division attempted with num2 = 0	Auto-reset after 1.5s
Overflow	Factorial input > 12 or result > 14 chars	Auto-reset after 1.5s

Flow Chart

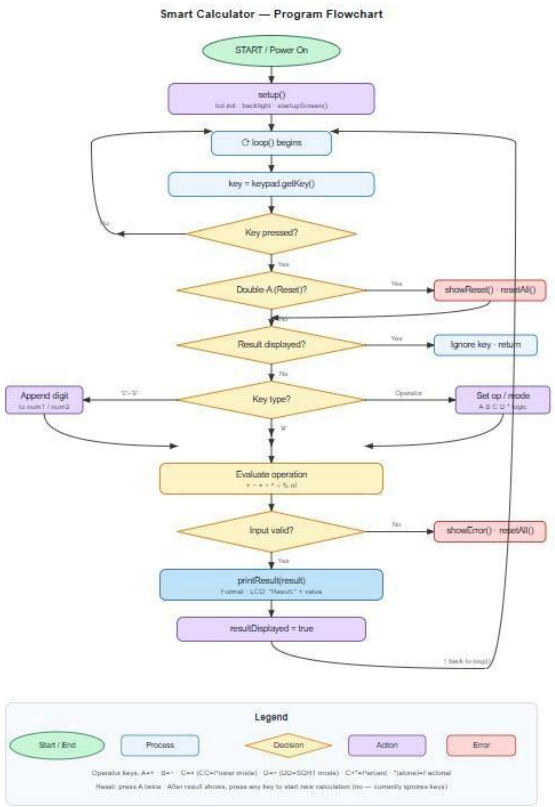


Fig : Flow Chart

METHODOLOGY

1. **Hardware Assembly**
   - Connect keypad rows to digital pins (5,4,3,2).
   - Connect columns to analog pins (A3, A2, A1, A0).
   - Connect LCD via I2C (SDA, SCL).
2. **Software Implementation**
   - Install required libraries: Wire.h, LiquidCrystal_I2C.h, Keypad.h.
   - Define keypad matrix mapping.
   - Implement factorial function.
   - Add mode control for percentage and power.
   - Include error handling.
3. **Testing**
   - Validate each arithmetic function.
   - Test overflow condition for factorial.
   - Test division by zero.
   - Verify power toggle mode.
4. **Debugging**
   - Monitor LCD outputs.
   - Adjust delay timings.
   - Optimize input handling.

User Directions & Operation Guide

Basic Operations

Addition (+)

1. Type your first number using the digit keys (0-9)
2. Press A - the display shows +
3. Type your second number
4. Press # to see the result

Example: 5 A 3 # → Result: 8

Subtraction (-)

1. Type your first number
2. Press B - the display shows -
3. Type your second number
4. Press # to see the result

Example: 9 B 4 # → Result: 5

Multiplication (x)

1. Type your first number
2. Press C once - the display shows x
3. Type your second number
4. Press # to see the result

Example: 6 C 7 # → Result: 42

Division (/)

1. Type your first number
2. Press D once - the display shows /
3. Type your second number
4. Press # to see the result

Example: 8 D 2 # → Result: 4

Factorial (!)

1. Type a number (must be 12 or less)
2. Press * - the display shows !
3. Press # to see the result

Example: 5 * # → Result: 120

Special Modes

The calculator has three special modes for advanced operations. Modes must be activated BEFORE entering numbers.

Power Mode (a^b)

1. Press C twice quickly - display briefly shows "Power Mode"
2. Type the base number
3. Press C once - the display shows ^ Type the exponent
4. Press # to see the result

Example: C C then 2 C 8 # → Result: 256

Square Root Mode (√a)

1. Press D twice quickly - display briefly shows "SQRT Mode"
2. Type the number
3. Press D once - the display shows sqr
4. Press # to see the result

Example: D D then 1 6 D # → Result: 4

Percent Mode (a x b / 100)

5. Type your first number
6. Press C, then press * - display briefly shows "Percent Mode" then restores the first number
7. Press D - the display shows %
8. Type the percentage value
9. Press # to see the result

Example: 2 0 C * then D 5 0 # → Result: 10 (50% of 20)

Resetting the Calculator

Press A twice in a row from anywhere to fully reset the calculator. The screen will briefly show "Reset" and all state, modes, and numbers will be cleared. This works at any point during a calculation.

Mode Quick Reference

Mode	Activation	Operator Key	Use Case
Normal	Default	A / B / C / D / *	+ , - , x , / , !
Power	Press C C	C	a to the power of b
SQRT	Press D D	D	Square root of a
Percent	(num) then C then *	D	a x b / 100

COMPONENTS REQUIRED / USED

1. Software Used

1. Thinkercad
2. Embedded C++
3. Libraries:
   - Wire.h
   - LiquidCrystal_I2C.h
   - Keypad.h
   - math.h

2. Hardware Components

Component	Specification
Arduino UNO R3	ATmega328P
Keypad	4x4 Matrix
LCD	16x2 I2C
Power Supply	5V USB

3. Components Overview

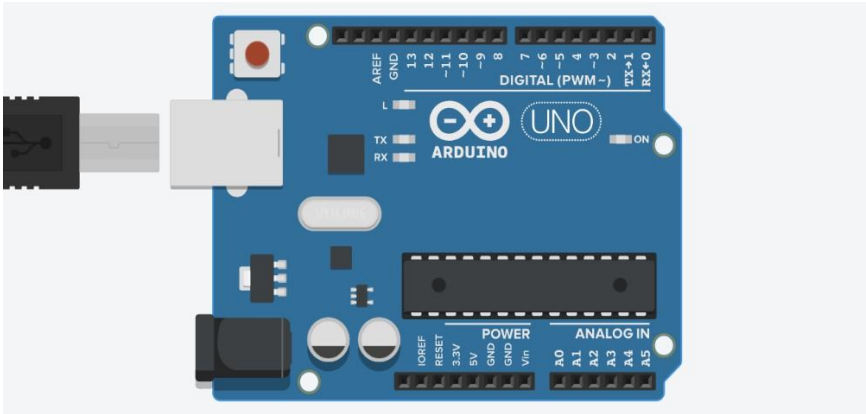


Fig : Arduino UNO R3

Fig : LCD Display

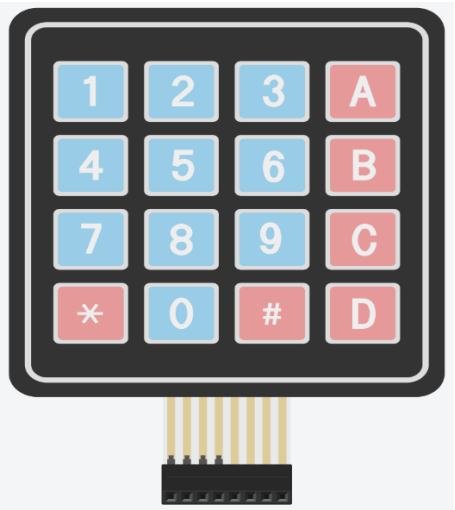


Fig : 4x4 Keypad

4. FINAL PROJECT VIEW

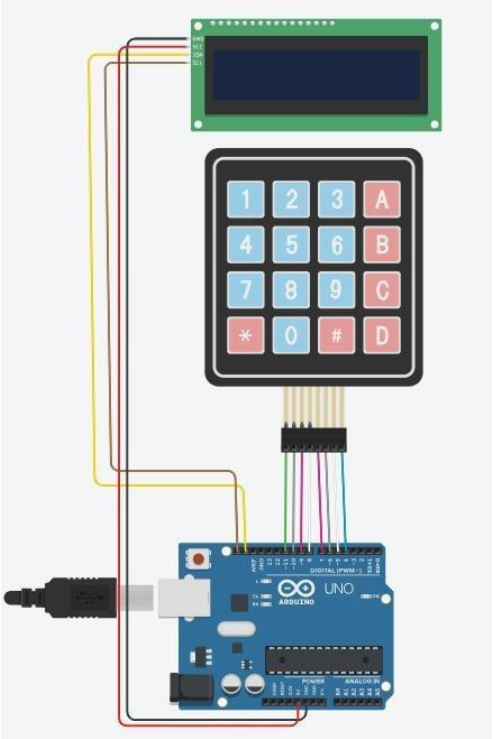


Fig : Final Project View

5. ATTACHED LINKS

- **Simulation Link:** TinkerCad
- **Git Repository Link:** GitHub

CONCLUSION

The Smart Calculator successfully demonstrates the integration of mathematics and embedded systems using Arduino. The system performs both basic and advanced arithmetic operations with reliable error handling.

This project contributes to **SDG 4 – Quality Education** by offering a low-cost educational tool that enhances programming knowledge and practical STEM exposure.

The project also strengthens understanding of:

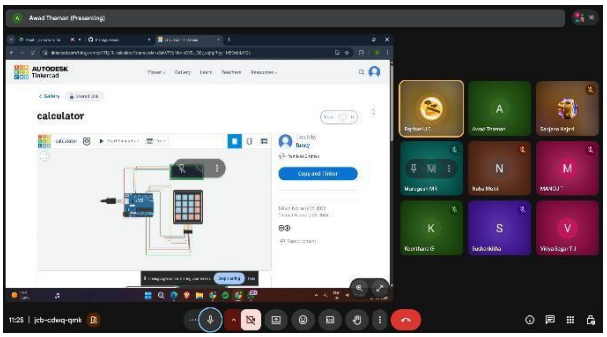
- Microcontroller interfacing
- Keypad matrix scanning
- LCD communication via I2C
- Mathematical function implementation
- Embedded error handling techniques

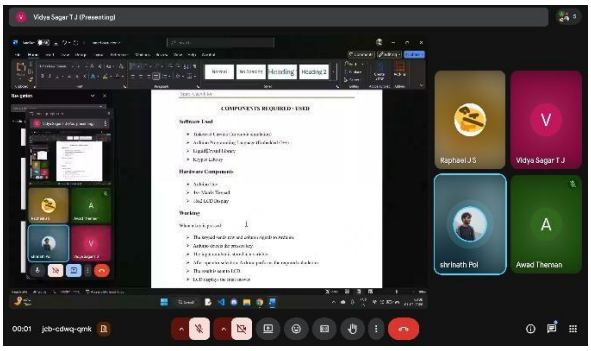
Future improvements may include:

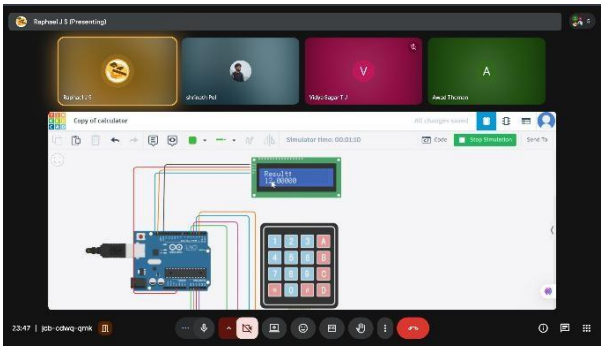
- Scientific functions (sin, cos, log)
- EEPROM memory storage
- Touchscreen interface
- Bluetooth connectivity

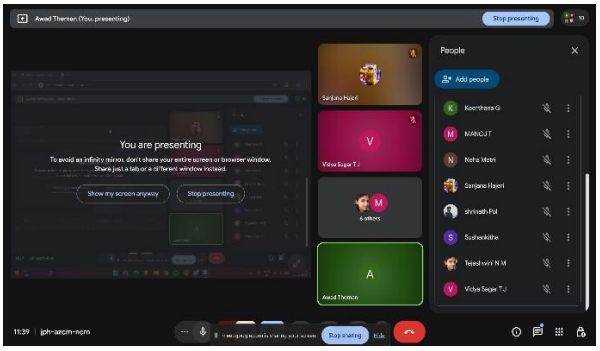
The project meets the operational and documentation standards specified in the IIMSTC guidelines and follows the required report structure.

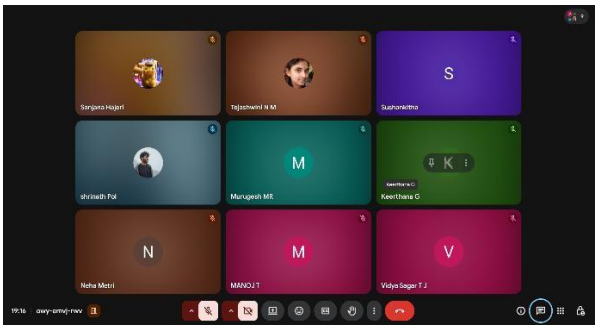
PROJECT DISCUSSION











Arduino Source Code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#include <math.h>

// Configuration & Global Variables
LiquidCrystal_I2C lcd(0x27, 16, 2);
const byte ROWS = 4;
const byte COLS = 4;
byte rowPins[ROWS] = {11, 10, 9, 8};
byte colPins[COLS] = {7, 6, 5, 4};

char keys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

Keypad keypad = Keypad(makeKeymap(keys),
rowPins, colPins, ROWS, COLS);

char num1[16] = "";
char num2[16] = "";
char op = 0;
byte idx1 = 0;
byte idx2 = 0;

bool factorialMode = false;
bool resultDisplayed = false;
bool firstAPressed = false;

enum Mode { NORMAL, POWER, PERCENT,
SQRT };
Mode currentMode = NORMAL;


// UI Helper Functions
void clearFirstRow() {
  lcd.setCursor(0, 0);
  for (int i = 0; i < 16; i++) {
    lcd.print(" ");
  }
  lcd.setCursor(0, 0);
}
void startupScreen() {
  lcd.clear();
  lcd.print("Smart Calculator");
  delay(1200);
  lcd.clear();
}
void showMode(const char msg[]) {
  clearFirstRow();
  lcd.print(msg);
  delay(800);
  lcd.clear();
}
void resetAll() {
  for (int i = 0; i < 16; i++) {
    num1[i] = '\0';
    num2[i] = '\0';
  }
  idx1 = 0;
  idx2 = 0;
  op = 0;
  factorialMode = false;
  resultDisplayed = false;
  currentMode = NORMAL;
  lcd.clear();
}
void showReset() {
  lcd.clear();
  lcd.print("Reset");
  delay(900);
  resetAll();
}
void showError(const char msg[]) {
  lcd.clear();
  lcd.print(msg);
  delay(1500);
  resetAll();
}
// Logic & Calculation Functions
void printResult(double result) {
  char buffer[16];

  // Handle formatting based on whether it is a
whole number or needs decimals
  if (result >= 0 && result == floor(result) &&
result <= 4294967295.0) {
    ultoa((unsigned long)result, buffer, 10);
  } else {
    result += 0.000001;
    result = round(result * 10000.0) / 10000.0;
    dtostrf(result, 0, 4, buffer);
  }
  // Check for overflow on the 16-character screen
  if (strlen(buffer) > 14) {
    showError("Overflow");
    return;
  }
  lcd.clear();
```

```
  lcd.print("Result:");
  lcd.setCursor(0, 1);
  lcd.print(buffer);
  resultDisplayed = true;
}
unsigned long factorial(unsigned long n) {
  unsigned long f = 1;
  for (int i = 1; i <= n; i++) {
    f *= i;
  }
  return f;
}
bool checkReset(char key) {
  if (key != 'A') {
    firstAPressed = false;
    return false;
  }
  if (!firstAPressed) {
    firstAPressed = true;
    return false;
  }
  showReset();
  firstAPressed = false;
  return true;
}
// Main Setup
void setup() {
  lcd.init();
  lcd.backlight();
  startupScreen();
}
// Main Loop
void loop() {
  char key = keypad.getKey();

  // Ignore input if nothing pressed, reset triggered,
or result already showing
  if (!key || checkReset(key) || resultDisplayed) {
    return;
  }
  // Handle Number Inputs
  if (key >= '0' && key <= '9') {
    if (op == 0) {
      if (idx1 < 14) {
        num1[idx1++] = key;
        num1[idx1] = '\0';
        lcd.print(key);
      }
    } else {
      if (idx2 < 14) {
        num2[idx2++] = key;
        num2[idx2] = '\0';
        lcd.print(key);
```

```
      }
    }
    return;
  }
  // Handle Basic Operators
  if (key == 'A' && currentMode == NORMAL) {
    op = 'A';
    lcd.print("+");
    return;
  }
  if (key == 'B' && currentMode == NORMAL) {
    op = 'B';
    lcd.print("-");
    return;
  }
  // Handle Multiplication & Power
  if (key == 'C') {
    if (op == 'C' && currentMode == NORMAL) {
      resetAll();
      currentMode = POWER;
      showMode("Power Mode");
      return;
    }
    if (currentMode == POWER) {
      op = 'P';
      lcd.print("^");
      return;
    }
    if (currentMode == NORMAL) {
      op = 'C';
      lcd.print("x");
      return;
    }
  }
  // Handle Division, Square Root, & Percent
  if (key == 'D') {
    if (op == 'D' && currentMode == NORMAL) {
      resetAll();
      currentMode = SQRT;
      showMode("SQRT Mode");
      return;
    }
    if (currentMode == SQRT) {
      op = 'S';
      lcd.print("sqr");
      return;
    }
    if (currentMode == PERCENT) {
      op = 'R';
      lcd.print("%");
      return;
    }
    if (currentMode == NORMAL) {
```

```
      op = 'D';
      lcd.print("/");
      return;
    }
  }
  // Handle Factorial & Percent Triggers
  if (key == '*') {
    if (op == 'C' && currentMode == NORMAL) {
      op = 0;
      currentMode = PERCENT;
      showMode("Percent Mode");
      lcd.print(num1);
      return;
    }
    if (op == 0 && currentMode == NORMAL) {
      factorialMode = true;
      lcd.print("!");
    }
    return;
  }
  // Calculate Results
  if (key == '#') {
    double a = atof(num1);
    double b = atof(num2);
    double result = 0;

    if (factorialMode) {
      if ((unsigned long)a > 12) {
        showError("Overflow");
        return;
      }
      printResult((double)factorial((unsigned
long)a));
      return;
    }

    if (op == 'S') { printResult(sqrt(a)); return; }
    if (op == 'P') { printResult(round(pow(a, b)));
return; }
    if (op == 'R') { printResult((a * b) / 100.0);
return; }

    if (op == 'A') {
      result = a + b;
    } else if (op == 'B') {
      result = a - b;
    } else if (op == 'C') {
      result = a * b;
    } else if (op == 'D') {
      if (b == 0) {
        showError("Div by 0");
        return;
      }
      result = a / b;
    }

    printResult(result);
  }
}
```